COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 212-Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	9/3/26	**DATE SUBMITTED:**	9/5/26
NAME:	Francis Nikko I. Andoy	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-On Activity 6.1 Targeting Specific Nodes

1. PROCEDURE OUTPUT

Task 1: Targeting Specific Nodes

```
TASK [Install updates (CentOS)] ************************************************************************
skipping: [ubuntu1]
skipping: [ubuntu2]
ok: [centos1]

TASK [install updates (Ubuntu)] ************************************************************************
skipping: [ubuntu1]
skipping: [ubuntu2]
skipping: [centos1]

PLAY [web_servers] *************************************************************************************

TASK [Gathering Facts] *********************************************************************************
ok: [ubuntu2]
ok: [ubuntu1]

TASK [Install apache and php for ubuntuservers] ********************************************************
ok: [ubuntu2]
ok: [ubuntu1]

TASK [Install apache and php for CentOSservers] ********************************************************
skipping: [ubuntu1]
skipping: [ubuntu2]

PLAY RECAP *********************************************************************************************
centos1                    : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0
   ignored=0
ubuntu1                    : ok=3    changed=0    unreachable=0    failed=0    skipped=3    rescued=0
   ignored=0
ubuntu2                    : ok=3    changed=0    unreachable=0    failed=0    skipped=3    rescued=0
   ignored=0
ubuntu3                    : ok=0    changed=0    unreachable=1    failed=0    skipped=0    rescued=0
   ignored=0
```

- **For this first task. The updated site.yml includes, updating and installing of software when the command was executed it first checks the connection of the control to the other nodes via ssh and using the publickey so I don't have to verify everytime. The ubuntu 3 is not on because I can't open 4 VM simultaneously.**

```
PLAY [db_servers] ***************************************************************************

TASK [Gathering Facts] **********************************************************************
ok: [centos1]

TASK [install mariadb package (CentOS)] *****************************************************
changed: [centos1]

TASK [Mariadv- Restarting/Enabling] *********************************************************
changed: [centos1]

TASK [install mariadb package (Ubuntu)] *****************************************************
skipping: [centos1]

PLAY RECAP **********************************************************************************
centos1                    : ok=5    changed=2    unreachable=0    failed=0    skipped=2    rescued=0
   ignored=0
ubuntu1                    : ok=3    changed=0    unreachable=0    failed=0    skipped=3    rescued=0
   ignored=0
ubuntu2                    : ok=3    changed=0    unreachable=0    failed=0    skipped=3    rescued=0
   ignored=0
ubuntu3                    : ok=0    changed=0    unreachable=1    failed=0    skipped=0    rescued=0
   ignored=0
```

- For this part the site.yml was edited to include the db_servers to be included. The process includes the installation and restarting then enabling of the said mariadb. The mariadb for ubuntu is optional in my part because I used the IP address of my centOS to db_servers so the ubuntu is skipped.

```
TASK [install samba package] ****************************************************************
changed: [ubuntu2]

PLAY RECAP **********************************************************************************
centos1                    : ok=5    changed=1    unreachable=0    failed=0    skipped=2    rescued=0
   ignored=0
ubuntu1                    : ok=3    changed=0    unreachable=0    failed=0    skipped=3    rescued=0
   ignored=0
ubuntu2                    : ok=3    changed=1    unreachable=0    failed=0    skipped=2    rescued=0
   ignored=0
```

- For this part the process is the same for the other parts. I just added the necessary - hosts which is the file_servers then give the something like sudo command. But the primary difference is that instead of using a package manager like dnf and apt I instead use package since it is the same for the both distros. I also observed that in the code it didn't use the when statement since again the package is same for the 2 distro.

Task 2: Using Tags in running playbooks

ansible-playbook --list-tags site.yml

```
(base) nikko@Nikko:~/CPE212_Andoy-/ansible/hoa4-yarn$ ansible-playbook --list-tags ../site.yml

playbook: ../site.yml

  play #1 (all): all    TAGS: []
      TASK TAGS: [always]

  play #2 (web_servers): web_servers    TAGS: []
      TASK TAGS: [apache, apache2, centos, httpd, ubuntu]

  play #3 (db_servers): db_servers      TAGS: []
      TASK TAGS: [centos, db, mariadb, ubuntu]

  play #4 (file_servers): file_servers  TAGS: []
      TASK TAGS: [samba]
```

ansible-playbook --tags centos --ask-become-pass site.yml

```
PLAY [file_servers] ************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [ubuntu2]

PLAY RECAP *********************************************************************************************************
centos1                    : ok=4    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
ubuntu1                    : ok=2    changed=0    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
ubuntu2                    : ok=2    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
```

ansible-playbook --tags db --ask-become-pass site.yml

```
TASK [install mariadb package (Ubuntu)] ****************************************************************************
skipping: [centos1]

PLAY [file_servers] ************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [ubuntu2]

PLAY RECAP *********************************************************************************************************
centos1                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
ubuntu1                    : ok=2    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
ubuntu2                    : ok=2    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0

(base) nikko@Nikko:~/CPE212_Andoy-/ansible/hoa4-yarn$
```

ansible-playbook --tags apache --ask-become-pass site.yml

```
PLAY [db_servers] **************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [centos1]

PLAY [file_servers] ************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [ubuntu2]

PLAY RECAP *********************************************************************************************************
centos1                    : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
ubuntu1                    : ok=3    changed=0    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
ubuntu2                    : ok=2    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
```

ansible-playbook --tags “apache,db” --ask-become-pass site.yml

```
PLAY [db_servers] ***********************************************************************************************************

TASK [Gathering Facts] ******************************************************************************************************
ok: [centos1]

PLAY [file_servers] *********************************************************************************************************

TASK [Gathering Facts] ******************************************************************************************************
ok: [ubuntu2]

PLAY RECAP ******************************************************************************************************************
centos1                    : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
ubuntu1                    : ok=2    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
ubuntu2                    : ok=2    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
```

Task 3: Managing Services

```
TASK [Install apache and php for ubuntu servers] ****************************************************************************
ok: [ubuntu1]

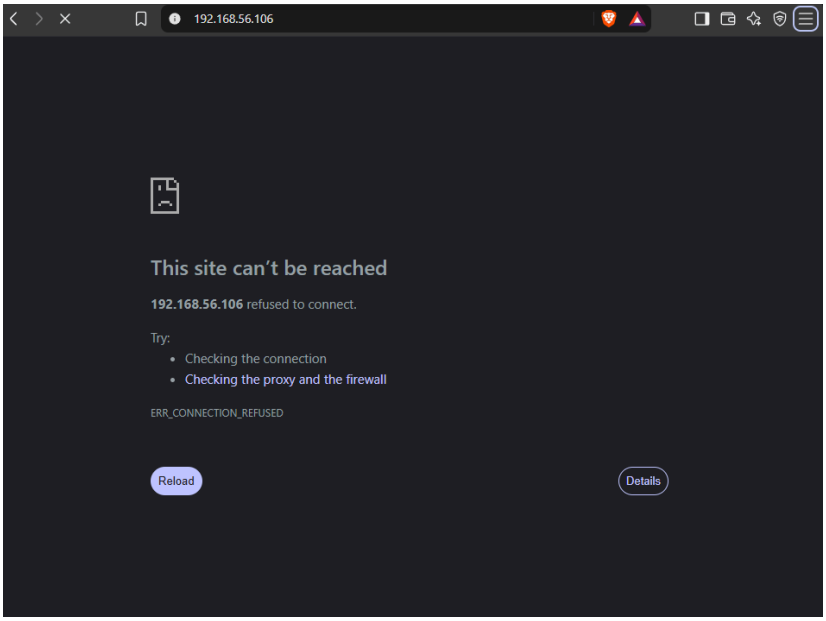
TASK [Install apache and php for CentOSservers] *****************************************************************************
skipping: [ubuntu1]

TASK [Start httpd (CentOS)] *************************************************************************************************
skipping: [ubuntu1]
```

- **For this it just starts the service of the httpd using the “service”**

To test it, before you run the saved playbook, go to the CentOS server and stop the currently running httpd using the command sudo systemctl stop httpd. When prompted, enter the sudo password. After that, open the browser and enter the CentOS server’s IP address. You should not be getting a display because we stopped the httpd service already.

```
[cent-server@localhost ~]$ sudo systemctl stop httpd
[cent-server@localhost ~]$
```



Go to the local machine and this time, run the site.yml file. Then after running the file, go again to the CentOS server and enter its IP address on the browser. Describe the result.
To automatically enable the service every time we run the playbook, use the command enabled: true similar to Figure 7.1.2 and save the playbook.



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary Activity:
1. Creating a Role for Web Servers
2. Creating a Role for Web Servers
3. Creating a Role for Database Servers
4. Combining Multiple Roles in a Playbook
To orchestrate multiple roles in a single playbook for different server groups.
Add tags to each role to allow selective execution, e.g., tags: web for webserver, tags: db for database.

Web tags:

```
(base) nikko@Nikko:~/CPE212_Andoy-/ansible$ ansible-playbook --tags db -K roles.yml
BECOME password:

PLAY [all] ********************************************************************************************************

TASK [Gathering Facts] ********************************************************************************************
[WARNING]: Host 'ubuntu1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation o
tps://docs.ansible.com/ansible-core/2.19/reference_appendices/interpreter_discovery.html for more information.
ok: [ubuntu1]
[WARNING]: Host 'ubuntu2' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation o
tps://docs.ansible.com/ansible-core/2.19/reference_appendices/interpreter_discovery.html for more information.
ok: [ubuntu2]
[WARNING]: Host 'centos1' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of
ps://docs.ansible.com/ansible-core/2.19/reference_appendices/interpreter_discovery.html for more information.
ok: [centos1]

TASK [update cache in all servers] ********************************************************************************
changed: [ubuntu2]
ok: [centos1]
changed: [ubuntu1]

PLAY [web_servers] ************************************************************************************************

TASK [Gathering Facts] ********************************************************************************************
ok: [centos1]

PLAY [db_servers] *************************************************************************************************

TASK [Gathering Facts] ********************************************************************************************
ok: [ubuntu1]

PLAY [file_servers] ***********************************************************************************************

TASK [Gathering Facts] ********************************************************************************************
ok: [ubuntu2]

PLAY RECAP ********************************************************************************************************
centos1                    : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu1                    : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu2                    : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

(base) nikko@Nikko:~/CPE212_Andoy-/ansible$
```

Db tags:

```
(base) nikko@Nikko:~/CPE212_Andoy-/ansible$ ansible-playbook --tags web -K roles.yml
BECOME password:

PLAY [all] ********************************************************************************************************

TASK [Gathering Facts] ********************************************************************************************
[WARNING]: Host 'ubuntu2' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of
tps://docs.ansible.com/ansible-core/2.19/reference_appendices/interpreter_discovery.html for more information.
ok: [ubuntu2]
[WARNING]: Host 'centos1' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of a
ps://docs.ansible.com/ansible-core/2.19/reference_appendices/interpreter_discovery.html for more information.
ok: [centos1]
[WARNING]: Host 'ubuntu1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of
tps://docs.ansible.com/ansible-core/2.19/reference_appendices/interpreter_discovery.html for more information.
ok: [ubuntu1]

TASK [update cache in all servers] ********************************************************************************
changed: [ubuntu2]
changed: [ubuntu1]
ok: [centos1]

PLAY [web_servers] ************************************************************************************************

TASK [Gathering Facts] ********************************************************************************************
ok: [centos1]

PLAY [db_servers] *************************************************************************************************

TASK [Gathering Facts] ********************************************************************************************
ok: [ubuntu1]

PLAY [file_servers] ***********************************************************************************************

TASK [Gathering Facts] ********************************************************************************************
ok: [ubuntu2]

PLAY RECAP ********************************************************************************************************
centos1                    : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu1                    : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu2                    : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Analysis:

- **Basically, I created a 3 separate child folder inside of the roles parent folder to organize the folder structure. I also added the tasks folder, supposedly the tasks can be used inside of the code but I discovered that there are a lot of tasks like function which can also be created/used by creating a folder. Since it will just sequentially move to the path where the tags live. I also used the when statement to run the script efficiently because it will just skip the distro when the value returned is false.**

- **To explain the processes of the roles.yml which contains the roles and tags. The first thing this script do is to update the cache of all the servers using the package. I didn't explicitly use the package manager of a distro because I want my script to be simple and easy to read, by using the package we can assume that all the package managers of every distro inside of the inventory will be updated. Next will be the servers. The servers just need the sudo like password, same for updating the cache. My understanding of the code is that roles is that it is like the tasks and pre_tasks and since I created the tasks folder I don't have to define it in the code. I did that for simplicity purposes and for a structured file system. And then for the role: web_servers this is the file/folder identifier and lastly is the tags which is for running the script individually and efficiently.**

Reflections:
Answer the following:

1. What is the importance of putting our remote servers into groups?
    - By putting them into groups we can simplify the script that we will create, we can also enhance the readability of our script by doing so. It also mirrors production infrastructure where multi-tier architectures demand distinct configuration logic per node category.
2. What is the importance of tags in playbooks?
    - Tags operate by attaching operational metadata to tasks, plays, or roles.
3. Why do you think some services need to be managed automatically in playbooks?
    - Since the default of some daemon of another distro is set to inactive and disabled we can lessen the work by always putting the service to ensure that our services will automatically run after downloading.

Conclusion:

- In conclusion, I've learned how to simplify my code for readability and clarity. When using tags we basically are using metadata to our tasks, and when we group our servers we mirrors production infrastructure to make it easily for us to automate things. I encountered a problem when finishing my supple where in the path is wrong.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.